

DbSet in Entity Framework Database First Approach with Examples

In this article, I am going to discuss **DbSet Class in Entity Framework Database First Approach with Examples**. Please read our previous article where we discussed [DbContext in Entity Framework](#). At the end of this article, you will understand the need and use of Entity Framework DbSet Class in detail. We are going to work with the same example that we created in our [Introduction to Entity Framework Database First](#) article. Please read our introduction to [Entity Framework Database First](#) article before proceeding to this article.

DbSet Class in Entity Framework

The DbSet Class in Entity Framework represents an Entity Set that can be used to perform the database CRUD Operations i.e. CREATE, SELECT, UPDATE, and DELETE operations. The context class should derive from [DbContext Class](#) and must include the DbSet type properties for the Entities which will map to database tables and views. The DbSet class in Entity Framework is actually an implementation of the [Repository Pattern](#).

Example to Understand DbSet Class in Entity Framework with Examples:

As we already discussed in our previous articles, the Entity Framework Database First Approach creates the following Context class for us based on the database tables, views, stored procedures, and stored functions. It includes all the model classes as DbSet<TEntity> type where TEntity is nothing but the model class which maps to either database tables or views. As you can see in the below image, for each database table and view, the entity framework creates DbSet properties.

```
namespace DBFirstApproach
{
    public partial class EF_Demo_DBEntities : DbContext Context class must be inherited from DbContext class
    {
        public EF_Demo_DBEntities()
            : base("name=EF_Demo_DBEntities")
        {
        }

        Fluent API
        protected override void OnModelCreating(DbModelBuilder modelBuilder)
        {
            throw new UnintentionalCodeFirstException();
        }

        Entity Set
        public virtual DbSet<Course> Courses { get; set; }
        public virtual DbSet<Standard> Standards { get; set; }
        public virtual DbSet<Student> Students { get; set; }
        public virtual DbSet<StudentAddress> StudentAddresses { get; set; }
        public virtual DbSet<Teacher> Teachers { get; set; }
        public virtual DbSet<vwStudentCourse> vwStudentCourses { get; set; }

        public virtual int spDeleteStudent(Nullable<int> studentId) {---}
        public virtual ObjectResult<spGetCoursesByStudentId_Result> spGetCoursesByStudentId(Nullable<int> studentID)
        {----}
        public virtual ObjectResult<Nullable<decimal>> spInsertStudent(Nullable<int> standardId, string firstName, string lastName)
        {-----}
        public virtual int spUpdateStudent(Nullable<int> studentId, Nullable<int> standardId, string firstName, string lastName)
        {-----}
    }
}
```

Note: A DbSet represents the collection of all entities in the context or you can say DbSet is a set or collection of entities where each entity is mapped to a database table or view. And then using this DbSet entity, we can perform the database CRUD Operations. The DbSet instance is created by calling the Set method on the DbContext object by passing the Entity Type as a parameter.

Methods of DbSet Class in Entity Framework:

The Entity Framework DbSet Class provides some methods using which we can perform different types of operations. Let us understand some of the commonly used methods of DbSet.

DbSet Add Method in Entity Framework:

The DbSet Add method is used to add the given entity to the context object with the Added State and once we add that entity is going to be tracked by the context object. And when we call the SaveChanges method, the entity which is in the Added state is inserted into the corresponding database table which is mapped with the given entity. Once the changes are saved into the database i.e. entity is inserted into the database or you can say once the SaveChanges method execution is completed successfully, the object state or entity state is moved from Added to Unchanged State.

Example to Understand DbSet Add Method:

For a better understanding of the DbSet Add method in Entity Framework, please have a look at the below image. Here, we are creating a student object and then passing that student object to the Add method (now the entity student is in Added State), and then we are calling the SaveChanges() method and once the save changes method execution is completed, then the entity state will change from Added to Unchanged state.

```
using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
{
    //Creating a new student
    var student = new Student
    {
        FirstName = "Hina",
        LastName = "Sharma",
        StandardId = 1
    };

    //Adding the Entity to the context
    context.Students.Add(student); //Entity in Added State
    context.SaveChanges(); //Entity in unchanged state
}
```

The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```
using System;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
{
    //Creating a new student
    var student = new Student
    {
        FirstName = "Hina",
        LastName = "Sharma",
        StandardId = 1
    };
    //Adding new student to the context object
    context.Students.Add(student);
    //Now the Entity State will be in Added State
    Console.WriteLine($"Before SaveChanges Entity State:
{context.Entry(student).State}");
    //Calling the SaveChanges method to store the newly added
entity into the database
    context.SaveChanges();
    //Once the SaveChanges Method Executed Successfully, the
entity state will changed to Unchanged
    Console.WriteLine($"After SaveChanges Entity State:
{context.Entry(student).State}");

    Console.ReadKey();
}
}
}
}

```

DbSet AddRange Method in Entity Framework:

The DbSet AddRange method in Entity Framework is used to add a given collection of entities to the context object with the Added State. When the SaveChanges method is called on the Context object, all the entities which are in the Added state are inserted into the corresponding database table which mapped the given entity. Once the entities are inserted into the database, the entity states are then changed from Added to Unchanged State.

Example to Understand DbSet AddRange Method in Entity Framework:

For a better understanding of how to use the DbSet AddRange Method in Entity Framework, please have a look at the below image. Here, first, we created a list of Student objects i.e. list of student entities. Then we pass this list of student objects (i.e. students) to the AddRange method and then we are calling the SaveChanges() method.

```

using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
{
    IList<Student> students = new List<Student>()
    {
        new Student(){FirstName = "Test1", LastName = "Test11", StandardId = 1},
        new Student(){FirstName = "Test2", LastName = "Test22", StandardId = 2},
        new Student(){FirstName = "Test3", LastName = "Test33", StandardId = 3}
    };

    //Adding the Entity to the context
    context.Students.AddRange(students); //Entities in Added State
    context.SaveChanges(); //Entities in unchanged state
}

```

The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```

using System;
using System.Collections.Generic;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Creating a new student collection
                List<Student> students = new List<Student>
                {
                    new Student(){ FirstName="Test1", LastName="Test11",
StandardId=1},
                    new Student(){ FirstName="Test2", LastName="Test22",
StandardId=2},
                    new Student(){ FirstName="Test3", LastName="Test33",
StandardId=3}
                };
                //Adding student collection to the context object
                context.Students.AddRange(students);
                foreach (var student in students)
                {
                    //Now the each entity state will be in Added State
                    Console.WriteLine($"Before SaveChanges Entity State:
{context.Entry(student).State}");
                }

                //Calling the SaveChanges method to Insert the entities into
the database
                context.SaveChanges();
            }
        }
    }
}

```

```

        //Once the SaveChanges Method Executed Successfully, the
entity state will changed to Unchanged
        foreach (var student in students)
        {
            //Now the each entity state will be in Unchanged State
            Console.WriteLine($"Before SaveChanges Entity State:
{context.Entry(student).State}");
        }
        Console.ReadKey();
    }
}
}
}
}
}

```

Output:

```

Before SaveChanges Entity State: Added
Before SaveChanges Entity State: Added
Before SaveChanges Entity State: Added
Before SaveChanges Entity State: Unchanged
Before SaveChanges Entity State: Unchanged
Before SaveChanges Entity State: Unchanged

```

DbSet Attach Method in Entity Framework:

This method is used to attach the given entity to the context object with the Unchanged state. That means calling the SaveChanges method on the context object will not affect the database.

Example to Understand DbSet Attach Method in Entity Framework:

For a better understanding of the DbSet Attach method in Entity Framework, please have a look at the below image. Here, first, we create a student object. Then we pass the student object to the Attach method, not to the Add method. The attach method will change the state of the student entity to Unchanged and then we call the SaveChanges method which does not affect the database. So, in this case, the entity will be tracked by the context object.

```

using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
{
    //Creating new student
    var student = new Student
    {
        FirstName = "Hina",
        LastName = "Sharma",
        StandardId = 1
    };

    //Attaching the entity
    context.Students.Attach(student); //Entity in unchanged state
    context.SaveChanges(); //Does not affect the database
}

```

The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```

using System;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Creating a new student
                var student = new Student
                {
                    FirstName = "Hina",
                    LastName = "Sharma",
                    StandardId = 1
                };
                //Attaching the new student to the context object
                context.Students.Attach(student);
                //Now the Entity State will be in Unchanged State
                Console.WriteLine($"Before SaveChanges Entity State:
{context.Entry(student).State}");
                //Calling the SaveChanges method to store the newly added
                entity into the database
                context.SaveChanges();
                //Once the SaveChanges Method Executed Successfully, the
                entity state will changed to Unchanged
                Console.WriteLine($"After SaveChanges Entity State:
{context.Entry(student).State}");
                Console.ReadKey();
            }
        }
    }
}

```

```
}  
}
```

DbSet AsNoTracking Method in Entity Framework:

It is an extension method where the returned entities will not be tracked by the **DbContext** object. This will be a significant performance boost for read-only entities. That means, if you are returning some data from the database only for viewing purposes, not for UPDATE and DELETE purposes, then you need to use the AsNoTracking method. This method returns a new query where the entities returned will not be tracked by the System.Data.Entity.DbContext object.

For a better understanding please have a look at the below image. Here, we are returning a list of students using the AsNoTracking method. This list of students will not be tracked by the DbContext object which will improve the performance of the application. In this case, the entities are going to be in Detached State. As the entities are not tracked by the context, so, we cannot perform update or delete operations on these entities.

```
using (var context = new EF_Demo_DBEntities())  
{  
    var studentList = context.Students.AsNoTracking().ToList<Student>();  
}
```

The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```
using System;  
using System.Linq;  
namespace DBFirstApproach  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())  
            {  
                var studentList = context.Students.AsNoTracking().ToList();  
                foreach (var student in studentList)  
                {  
                    //Now the each entity state will be in Detached State  
                    Console.WriteLine($"Entities with AsNoTracking Entity  
State: {context.Entry(student).State}");  
                }  
  
                //Modifying the entities  
                foreach (var student in studentList)  
                {  
                    student.FirstName = student.FirstName + "Changed";  
                }  
                Console.WriteLine("");  
                foreach (var student in studentList)  
                {
```

```
//Now the each entity state will be in Detached State
Console.WriteLine($"After Modifying Entity State:
{context.Entry(student).State}");
}

//Calling the SaveChanges method doesnot affect the database
context.SaveChanges();
Console.WriteLine("");
foreach (var student in studentList)
{
    //Now the each entity state will be in Detached State
    Console.WriteLine($"After SaveChanges Entity State:
{context.Entry(student).State}");
}
Console.ReadKey();
}
}
}
```

Output:

[illegible]

DbSet Create Method in Entity Framework with Example:

The DbSet Create method in Entity Framework is used to create and return a new instance of the given entity. The newly created entity is not added or attached to the context object. Here, the entity will be created with the default values based on the data types of the properties. For a better understanding, please have a look at the below image. Here, the Student Entity is going to be created and will be in Detached State which is not going to be tracked by the context object.

```
using (var context = new EF_Demo_DBEntities())
{
    var newStudentEntity = context.Students.Create();
}
```

The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```
using System;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Creating new entity using the DbSet Create Method
                //The entity will be created with the default valued based on
                the data type of properties
                //Entity will be in detached state and hence will not be
                tracked by the context object
                var newStudent = context.Students.Create();
                //Now the entity state will be in Detached State
                Console.WriteLine($"Entity State with Create() Method:
                {context.Entry(newStudent).State}");

                //Calling the SaveChanges method doesnot affect the database
                context.SaveChanges();
                Console.WriteLine($"After SaveChanges Entity State:
                {context.Entry(newStudent).State}");

                Console.ReadKey();
            }
        }
    }
}
```

Output:

Entity State with Create() Method: Detached After SaveChanges Entity State: Detached

DbSet Include Method in Entity Framework:

The DbSet Include method in Entity Framework is used to specify the related objects to include in the query results. That means it returns the included non-generic LINQ to the Entities query against a DbContext. For a better understanding, please have a look at the below image. In the below query, it will return the student list along with their address. This is one of the most useful methods in Entity Framework when you are working with real-time applications. Here, you can specify the related entity which you want to include as part of the result set with the name as well as with the lambda expression as shown in the below image.

```
using (var context = new EF_Demo_DBEntities())
{
    var studentList1 = context.Students.Include("StudentAddress").ToList<Student>();
    var studentList2 = context.Students.Include(s => s.StudentAddress).ToList<Student>();
}
```

The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```
using System;
using System.Linq;
using System.Data.Entity;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Include(): Specifies the related objects to include in the
                query results.
                // var StudentList1 = context.Students.Include(x =>
                x.StudentAddress).ToList();
                var StudentList1 =
                context.Students.Include("StudentAddress").ToList();
                foreach (var student in StudentList1)
                {
                    Console.WriteLine($"{student.FirstName}
{student.LastName} {student.StudentAddress?.Address1}
{student.StudentAddress?.Address2}");
                }
                Console.ReadKey();
            }
        }
    }
}
```

In this case, it will create a SELECT query with Joins and retrieve the data from both Student and StudentAddress tables. The following is the SELECT query which is generated and executed by the context class. You can capture the query using the SQL Server Profiler tool.

```
SELECT
[Extent1].[StudentId] AS [StudentId],
[Extent1].[FirstName] AS [FirstName],
[Extent1].[LastName] AS [LastName],
[Extent1].[StandardId] AS [StandardId],
[Extent2].[StudentId] AS [StudentId1],
[Extent2].[Address1] AS [Address1],
[Extent2].[Address2] AS [Address2],
[Extent2].[Mobile] AS [Mobile],
[Extent2].[Email] AS [Email]
FROM [dbo].[Student] AS [Extent1]
LEFT OUTER JOIN [dbo].[StudentAddress] AS [Extent2] ON [Extent1].[StudentId] = [Extent2].[StudentId]
```

Example without Using DbSet Include Method:

Even though we can get the results without using the Include method. In the following example, I am not using the Include method, but still, I am getting the result as expected. In this case, it will first execute the Select Query against the Student database table and while accessing the student address it will issue another select query and this is for each entity.

```
using System;
using System.Linq;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                var StudentList1 = context.Students.ToList();
                foreach (var student in StudentList1)
                {
                    Console.WriteLine($"{student.FirstName}
{student.LastName} {student.StudentAddress?.Address1}
{student.StudentAddress?.Address2}");
                }
                Console.ReadKey();
            }
        }
    }
}
```

So, in this case, one SELECT query to retrieve all the records from the Students table and n number of select queries to retrieve the data from the StudentAddress table where n represents the number of entities or number of records retrieved from the Student table. Then you can imagine, if your database table contains a huge number of records, then many time round trips it needs to perform the database server to get the data. With the DbSet Include method, with a single SELECT statement using Join, it will retrieve all the records from the database which will improve the overall applications performance.

Note: It is always recommended to use the Include method if you want to retrieve the related entities. If related entities are not required, then don't use the Include method.

Including Multiple Entities using DbSet Include Method in Entity Framework:

Paths are all-inclusive. For example, if an include call indicates Include("Standard.Teachers), not only the Teachers are included, but also Standard is included. When you call the Include method, the query path is only valid on the returned instance of the DbQuery<T>. Other instances of DbQuery<T> and the object context itself are not affected. Because the Include method returns the query object, you can call this method multiple times on a DbQuery<T> to specify multiple paths for the query. For a better understanding, please have a look at the following example. This is not an ideal example, but for the understanding purpose, I am giving you this example.

```
using System;
using System.Linq;
using System.Data.Entity;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Using name
                //var studentsMS = context.Students
                //                .Include("Standard.Teachers").ToList();

                //Using Lambda Expression
                var studentsList = context.Students
                    .Include(s => s.Standard.Teachers).ToList();
                foreach (var student in studentsList)
                {
                    Console.WriteLine($"Student: Firstname:
{student.FirstName}, Lastname: {student.LastName}, StandardName:
{student.Standard.StandardName}, Description:
{student.Standard.Description}");
                    foreach (var teacher in student.Standard.Teachers)
                    {
                        Console.WriteLine($" \t Teacher: {teacher.FirstName}
{teacher.LastName}");
                    }
                }
                Console.ReadKey();
            }
        }
    }
}
```

DbSet Find Method in Entity Framework:

The DbSet Find method in Entity Framework is used to Find and return an entity with the given primary key values. If an entity with the given primary key values exists in the context, then it is returned immediately without making a request to

the database. Otherwise, a request is made to the database for an entity with the given primary key values and this entity, if found, is attached to the context and returned. If no entity is found in the context or in the database, then null is returned. For a better understanding, please have a look at the below image.

```
using (var context = new EF_Demo_DBEntities())
{
    Student student = context.Students.Find(5);
}
```

The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```
using System;
using System.Linq;
using System.Collections.Generic;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //return all students whose id is less than 5, Database Call
                List<Student> studentList = context.Students.Where(x =>
x.StudentId <= 5).ToList();
                foreach (var student in studentList)
                {
                    Console.WriteLine($"Id: {student.StudentId}, Firstname:
{student.FirstName}, Lastname: {student.LastName}");
                }
                //It will return the data from the context, not from the
database
                Student studentId3 = context.Students.Find(3);
                //It will return the data from the database, not from the
context
                Student studentId6 = context.Students.Find(6);

                //It will first check context and then database and finally
return null
                Student studentId10 = context.Students.Find(10);
                Console.ReadKey();
            }
        }
    }
}
```

In the above example, for retrieving all the students where student id <= 5, it will make a database call and store the result in the context object. Then when we use the Find method to fetch the student details whose Id = 3, it will check the context object and it will find the student details in the context object, and hence it will return the data from the

context object. Next, when we search the student details whose id = 6, in this case, it will check the context object and it will not find the student details there, and then it will go to the database and will fetch the Student details from the database. And, for the last Find method, it will check both context and database and return null. You can verify the same using SQL Server Profiler tool as shown in the below image.

EventClass	TextData	ApplicationName	NTUserName	LoginName
Audit Login	-- network protocol: LPC set quoted...	EntityFramework	Pranaya	LAPTOP...
SQL:BatchStarting	SELECT [Extent1].[StudentId] A...	EntityFramework	Pranaya	LAPTOP...
SQL:BatchCompleted	1 SELECT [Extent1].[StudentId] A...	EntityFramework	Pranaya	LAPTOP...
Audit Logout		EntityFramework	Pranaya	LAPTOP...
RPC:Completed	exec sp_reset_connection	EntityFramework	Pranaya	LAPTOP...
Audit Login	-- network protocol: LPC set quoted...	EntityFramework	Pranaya	LAPTOP...
RPC:Completed	2 exec sp_executesql N'SELECT TOP (2) ...	EntityFramework	Pranaya	LAPTOP...
Audit Logout		EntityFramework	Pranaya	LAPTOP...
RPC:Completed	exec sp_reset_connection	EntityFramework	Pranaya	LAPTOP...
Audit Login	-- network protocol: LPC set quoted...	EntityFramework	Pranaya	LAPTOP...
RPC:Completed	3 exec sp_executesql N'SELECT TOP (2) ...	EntityFramework	Pranaya	LAPTOP...

1. The First Select Statement Returns the data where StudentId <= 5
 2. The Second Select Statement Returns data where Student Id = 6
 3. The Third Select Statement Returns data where Student Id = 10

Note: There is no SQL Query for StudentId = 3

The point that you need to remember is that DbSet Find Method in Entity Framework also returns the entities that have been added to the context object but have not yet been saved to the database.

DbSet Remove Method in Entity Framework:

The DbSet Remove method is used to remove or delete an entity from the database. The point that you need to keep in mind is that the entity is not removed immediately rather the entity state will be marked as deleted. As soon as the SaveChanges method is called on the Context object, the entity is deleted from the database and the state is changed to Detached which means it is no longer tracked by the context object. The entity must exist in the context of some other state before this method is called. For a better understanding, please have a look at the below image.

```
using (var context = new EF_Demo_DBEntities())
{
    //First fetch the student data
    var student = context.Students.FirstOrDefault(x => x.StudentId == 5);

    //if the student exists then remove
    if (student != null)
    {
        context.Students.Remove(student);
        context.SaveChanges();
    }
}
```

The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```
using System;
using System.Linq;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //First Fetch the Student whose Id is 5
                Student student = context.Students.FirstOrDefault(x =>
x.StudentId == 5);
                //At this point the entity is in Unchanged State
                Console.WriteLine($"Entity State Before Remove Method:
{context.Entry(student).State}");
                //check if student exists then remove
                if (student != null)
                {
                    context.Students.Remove(student);
                    //At this point the entity moved to deleted state
                    Console.WriteLine($"Entity State After Remove Method:
{context.Entry(student).State}");
                    context.SaveChanges();
                    //At this point the entity moved to detached state and
entity will be deleted from the database
                    Console.WriteLine($"Entity State After SaveChanges
Method: {context.Entry(student).State}");
                }
                Console.ReadKey();
            }
        }
    }
}
```

Output:

```
Entity State Before Remove Method: Unchanged
Entity State After Remove Method: Deleted
Entity State After SaveChanges Method: Detached
```

DbSet RemoveRange Method in Entity Framework:

The DbSet RemoveRange Method in Entity Framework is used to remove or delete a collection of entities from the database. Like the DbSet Remove method, when the DbSet RemoveRange method is called, the entities are not removed immediately rather the entity's states will be marked as deleted. As soon as the SaveChanges method is called on the Context object, the entities are going to be deleted from the database, and after successful deletion, the


```

        {
            Console.WriteLine($"Entity State After RemoveRange
Method: {context.Entry(student).State}");
        }
        //Call SaveChanges Method to Delete these entities from
the database
        context.SaveChanges();
        //At this point the entity moved to detached state and
entity will be deleted from the database
        foreach (var student in ListStudents)
        {
            Console.WriteLine($"Entity State After SaveChanges
Method: {context.Entry(student).State}");
        }
    }
    Console.ReadKey();
}
}
}
}
}

```

Output:

```

Entity State Before RemoveRange Method: Unchanged
Entity State Before RemoveRange Method: Unchanged
Entity State Before RemoveRange Method: Unchanged
Entity State After RemoveRange Method: Deleted
Entity State After RemoveRange Method: Deleted
Entity State After RemoveRange Method: Deleted
Entity State After SaveChanges Method: Detached
Entity State After SaveChanges Method: Detached
Entity State After SaveChanges Method: Detached

```

DbSet SqlQuery Method in Entity Framework:

The DbSet SqlQuery Method in Entity Framework allows us to create a raw SQL query that will return entities. By default, the entities which are returned by the SqlQuery method are tracked by the context object. If you want you can change this default behavior by using the AsNoTracking method. For a better understanding, please have a look at the below image.

```

using (var context = new EF_Demo_DBEntities())
{
    var studentEntity = context.Students.SqlQuery("select * from Student where studentid = 5")
        .FirstOrDefault<Student>();
}

```

The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```

using System;

```

```

using System.Collections.Generic;
using System.Linq;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //First Fetch the List of Students to Remove
                Student student1 = context.Students.SqlQuery("SELECT * FROM
Student WHERE StudentId = 2").FirstOrDefault<Student>();
                Console.WriteLine($"Entity State of SqlQuery Method:
{context.Entry(student1).State}");
                //If you don't want to track the result of SQL Query then
you need to use AsNoTracking Method
                Student student2 = context.Students.SqlQuery("SELECT * FROM
Student WHERE StudentId = 3").AsNoTracking().FirstOrDefault<Student>();
                Console.WriteLine($"Entity State of SqlQuery Method:
{context.Entry(student2).State}");

                Console.ReadKey();
            }
        }
    }
}

```

Output:

```

Entity State of SqlQuery Method: Unchanged
Entity State of SqlQuery Method: Detached

```

In the next article, I am going to discuss the [Entity Framework Relationship](#) in detail with Examples. In this article, I try to explain the **DbSet Methods in Entity Framework Database First Approach** with Examples and I hope you enjoyed the DbSet Methods in Entity Framework with Examples article. Please give your valuable feedback and suggestions about this article.